

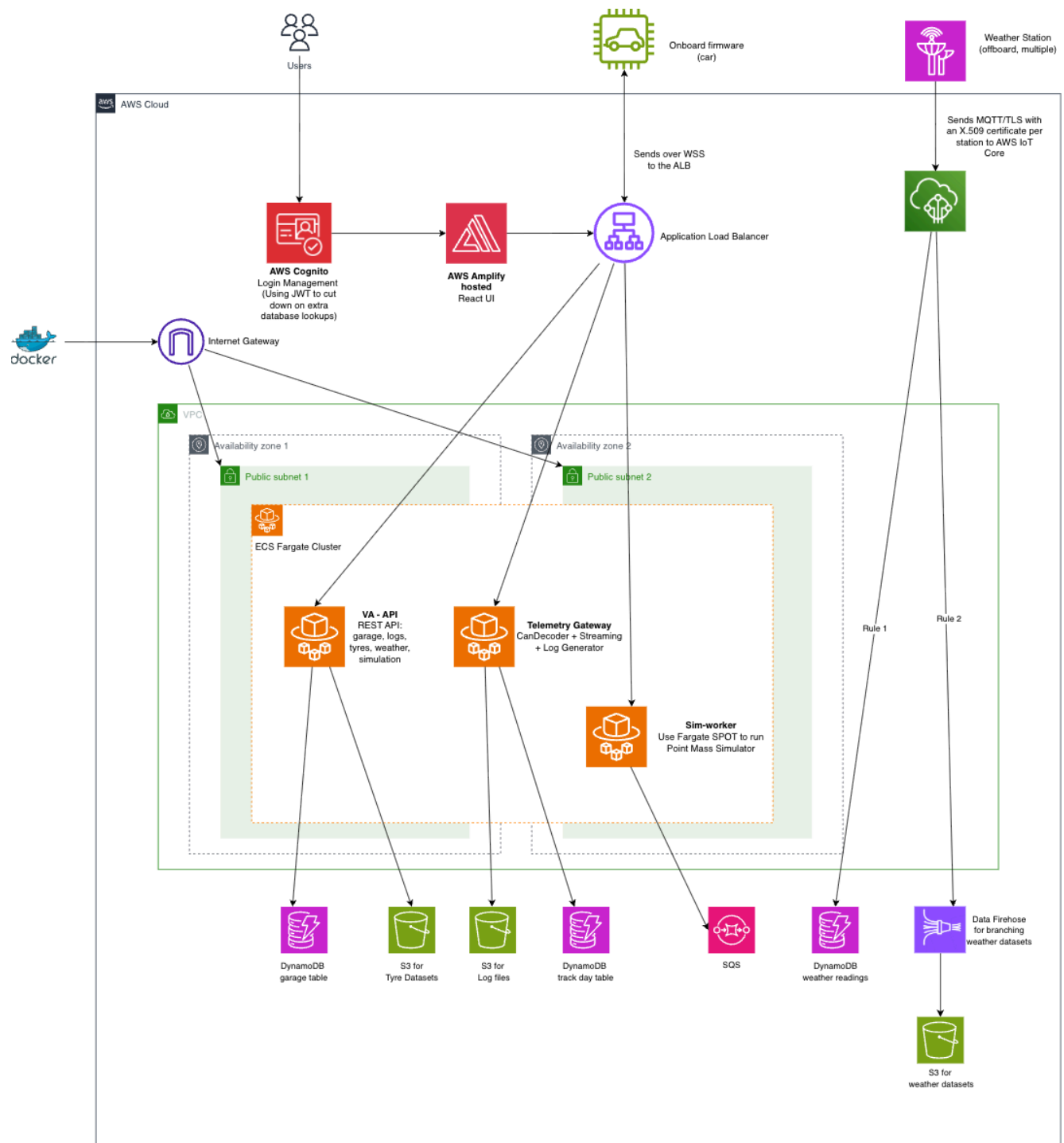
Redback Racing: Vehicle Analytics Cloud Technical Assessment

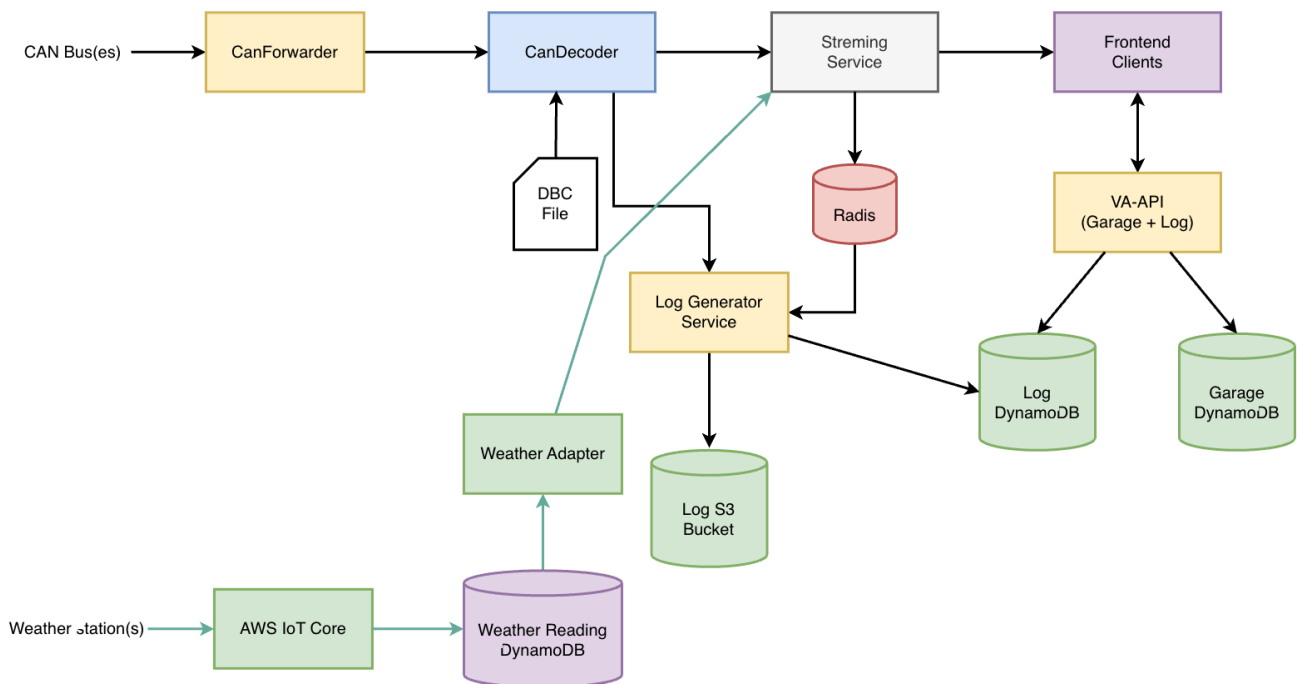
Maggie Wang - z5773706

Note: This assessment used some AI as technical assistance

1. Summary

The previous architecture diagram was a fairly complicated AWS microservice architecture structure as it contains lots of intermediary AWS services to connect the services needed for the AWS-based Vehicle Analytics cloud streaming/processing services. The new diagram has been redesigned to implement a local weather station collection system and achieve the goal of mitigating costs and considering minimising the complexity at the same time to improve the whole services structure. After the improvements, the new cloud architecture diagram and the system diagram look like this:





For clearer version of the diagrams, please check in the folder “submission” in the forked repository:

<https://github.com/MaggieW1121/va-cloud-assessment.git>

2. Change log: what has been changed and why

In consideration of costs, complexity, number of deployable units, precision the major elements changed in the cloud structures are...

- **The UI (Legacy) in the previous ECS fargate cluster has been removed:** Amplify is now the only frontend. This avoids maintaining two UIs and gives Git branch/PR previews with no per-seat charge (refer to the [AWS price list](#)).
- **Two API Gateways, VPC Link, and both Cloud Maps replaced by one public ALB:** Path rules send `/v1/*` to va-api and `/ws` and `/v1/logging/*` to telemetry-gateway. The car's and dashboards' WebSockets both need to reach the cluster, and API Gateway HTTP APIs can't proxy WebSockets to ECS, so to reduce the complexity the added layers have been removed.
- **CanDecoder, streaming-service, log generator, and Redis merged into one telemetry-gateway:** An in-process latest-value store replaces Redis, removing a network hop, an always-on cache, and a failure mode. This works because only one car (RB24) streams at a time, and Valkey can return behind an interface for competition-day resilience or concurrent sessions.
- **Garage-service and the Tyre, Garage, and Log management Lambdas combined into one va-api Fargate service:** They are all small CRUD/read endpoints, so one image, one Dockerfile, and one OpenAPI file is easier for new team members to learn.
- **S3-triggered Log Service Lambda dropped:** The log generator already knows the file, track, and session, so it writes the catalog entry itself. This removes the S3 event and a second place to debug.
- **Point Mass Simulator Lambda replaced by a sim-worker on Fargate Spot:** It scales from 0 to N on SQS queue depth. Replayable batch work suits Spot (up to about 70% cheaper), and SQS with a DLQ handles retries after interruptions.
- **Socket.IO and raw WebSocket unified:** One raw WebSocket protocol (NanoPB in from the car, MsgPack out to dashboards) means one protocol to document and test.
- **Cognito added:** It issues a JWT that va-api and the WebSocket connect step validate, which cuts down on extra database lookups.
- **Weather station pipeline added:** Stations connect to AWS IoT Core over MQTT/TLS with per-station X.509 certificates. IoT Rules write readings to DynamoDB and, via Firehose, to S3 archives. No compute is needed and there is no public write API, which keeps the attack surface small.

3. Weather station interpretations

I added a weather station pipeline that keeps the write path off the public API and needs no custom computation. Each station connects to AWS IoT Core over MQTT/TLS using its own X.509 certificate, publishing JSON to its own topic (weather/<stationId>/data), so identity is managed and can be revoked per device. Two IoT Rules then fan the data out: one writes each reading to the DynamoDB weather_readings table for "latest" and time-window queries, and the other sends the raw message through Kinesis Data Firehose, which batches and gzips it into an S3 archive that Athena can query later. To connect weather to the existing telemetry flow, I added a small weather adapter inside the telemetry-gateway that polls the latest reading about every 5 seconds and injects it as extra channels (for example Weather.AirTemp) from a reserved ID range. This means the Streaming module pushes weather to dashboards with no frontend protocol change, and the Log Generator writes weather columns into the same log CSV, so every recorded session includes the conditions it was driven in. Finally, va-api exposes /v1/weather/latest, /v1/weather/history and /v1/logs/{id}/weather so the team can read current and historical conditions and match them to a session afterwards.

4. Alternatives considered

- **Keep API Gateway in front (Amplify to API GW to VPC Link V2 to ALB).** Supported and reasonable, but the WebSocket path bypasses it, so it is a second front door; its useful parts (JWT auth, throttling) are available in the application. Reconsider if Redback wants managed throttling or usage plans.
- **IoT Core to SQS to a Fargate weather worker.** A container running 24/7 only to copy JSON into DynamoDB. IoT Rules do it without computer.
- **Six or more small ECS services (one per former function).** More deployables than the original. Consolidated by responsibility instead.
- **Replace Redis with DynamoDB.** DynamoDB is not a pub/sub or low-latency shared cache. Removal works only because the gateway is a single process; scale-out needs Redis/Valkey back.
- **Vercel for the frontend.** A good product, but a second platform and per-developer seats on paid tiers; Amplify keeps everything in one AWS account and has no per-seat charge.
- **Private subnets with NAT.** Stronger isolation, higher fixed cost. Recommended if the team later obtains sponsor credits or handles sensitive data.

Illustrative IaC (conceptual)

```
resource "aws_iot_topic_rule" "weather_to_ddb" {
  sql = "SELECT *, topic(2) AS stationId, timestamp() AS ts FROM
  'weather/+/data'" dynamodbv2 { role_arn = aws_iam_role.iot_rule.arn put_item {
    table_name =
    aws_dynamodb_table.weather.name } }
}
resource "aws_iot_topic_rule" "weather_to_s3" {
  sql = "SELECT * FROM 'weather/+/data'"
  firehose { delivery_stream_name = aws_kinesis_firehose_delivery_stream.weather.name role_arn =
  aws_iam_role.iot_rule.arn }
}
resource "aws_ecs_service" "sim_worker" { capacity_provider_strategy { capacity_provider =
"FARGATE_SPOT" weight = 1 } }
```

5. Other Considerations (debatable)

The considerations below are the uses of AWS services that I personally think can be changed based on the team's demands, as those services have their own advantages. In my case, I'm more so considering cost and my assumed racing contexts.

● WWS vs HTTP

HTTP follows a request-response model. The client opens a connection, sends a request, the server replies, and the exchange ends. To get fresh data, the client has to keep asking (polling), and every request carries its own headers and often a new connection handshake. Because the server can only answer when asked, any new data sits waiting until the next poll.

WSS (WebSocket Secure) starts with an HTTP request (HTTP handshake) that is upgraded into a single, long-lived, encrypted two-way connection. Once it is open, either side can send messages at any time without new handshakes or repeated headers. The server does not wait for a request; it pushes each new frame the moment it is available.

HTTP sends hundreds of bytes of header data with every single request. Repeated requests create high network overhead. WebSocket eliminates repetitive headers after the initial handshake. Framing overhead is more tiny (2 to 6 bytes), making it ideal for rapid data streams.

However, in terms of managing Cloud Infrastructure, HTTP has an advantage in lower cost per request. In comparison, WSS is more expensive because its cost is calculated from the connection duration + data amounts. So even though I chose to use WSS in this case, the Redback team may adjust the choice of services based on the cases.

● MQTT vs HTTP

HTTP follows a request-response model. The client opens a connection, sends a request, the server replies, and the exchange ends. For a weather station to report readings, it would have to send a fresh POST every time, and each request carries its own headers, often with a new TLS handshake. That means the station needs a public write endpoint, and the server has no built-in way to know whether the station is still alive or to deal with a message that failed to arrive.

MQTT is a lightweight publish/subscribe protocol that runs over a single, long-lived TLS connection. The device connects once to a broker (AWS IoT Core), then publishes small messages to a topic (for example `weather/<stationId>/data`). The broker routes each message to whatever is subscribed, such as an IoT Rule, so the sender never needs to know who consumes the data. Once connected, there are no repeated handshakes or large headers.

HTTP sends hundreds of bytes of header data with every request, which adds up on a device that reports every 5 to 10 seconds, and it drains more power and bandwidth on a small field station. MQTT's fixed header is only about 2 bytes, so it is well suited to small, frequent sensor readings on limited hardware. MQTT also has built-in quality-of-service levels (QoS 0 and 1 on IoT Core) so delivery can be acknowledged and retried, and Last Will messages can signal when a station drops offline, which HTTP does not offer natively.

In terms of network issues, since weather stations may encounter poor or even no network. MQTT was designed for this kind of low-bandwidth, high-latency, unreliable link. It keeps one long-lived connection with a keep-alive heartbeat and reconnects automatically after a drop, without repeating a full handshake for every message. Its messages are tiny (a 2-byte fixed header), so a reading can get through even on a poor signal. QoS 1 makes IoT Core acknowledge each message, and the station resends any that were not acknowledged, so a short outage does not silently lose data. A Last Will message can also flag when a station goes offline. HTTP has to open a new connection and TLS handshake for each request, which is slower and more likely to fail on a bad link, and the client has to build its own retry logic. MQTT is not magic, though. During a long outage the station must store readings locally and send them on reconnect, and QoS 1 can deliver duplicates, so each reading should carry the station's own timestamp to keep the DynamoDB records correctly ordered and to let duplicates be detected. That made up the main reason why I chose to use MQTT.

Security and identity also differ. With MQTT on IoT Core, each station has its own X.509 certificate and an IoT policy that only allows publishing to its own topic, so a device can be revoked individually and there is no public write API to attack. With HTTP, you would need to build and secure that endpoint yourself (API keys or tokens, throttling, validation).

However, in terms of managing cloud infrastructure, HTTP has an advantage in cost and simplicity. API Gateway and Lambda bill per request and need no always-on connection, while IoT Core bills per message (about 5 KB per metered unit) plus connection minutes, so a high message rate costs more. At one message per 10 seconds, the IoT Core cost is negligible, but at one per second it reaches roughly 2.6 million messages a month. So I chose MQTT because it is lighter, safer, and lets IoT Rules write to DynamoDB and S3 with no compute, but if the Redback team wants the simplest setup or has a station that can only speak HTTPS, they can use HTTP (or a small bridge that republishes to MQTT) instead.

● Fargate SPOT vs Lambda

Lambda is a serverless, event-driven computer service. When uploading a function, AWS runs it only when something triggers it (for example, an SQS message), then shuts it down. You are billed per invocation and per millisecond of execution, so a system that is idle most of the time costs almost nothing. The limits are a 15-minute maximum run time, a fixed memory and CPU ceiling, and a separate packaging and deployment model (zip or container image) from the rest of the system.

Fargate Spot runs ordinary containers on AWS's spare capacity at a discount of up to about 70% compared with on-demand Fargate. The container is the same image you run in docker compose, and it can run as long as the job needs. The catch is that AWS can reclaim the capacity with a two-minute warning, so tasks can be interrupted. Fargate bills for allocated CPU and memory for as long as a task is running, not per invocation.

For the Point Mass Simulator, Lambda's per-invocation billing is often cheaper for sporadic, tiny jobs, so Fargate is not chosen on price alone. The advantage of Fargate Spot is consistency and headroom. The simulator runs in the same container runtime as va-api and telemetry-gateway, so there is one Dockerfile, one local workflow (docker compose) and fewer deployable units to learn. There is also no 15-minute cap, which helps if longer simulation runs are needed, and CPU and memory can be sized freely for heavier computers.

The Spot risk is manageable because the work is replayable batch processing. The sim-worker reads jobs from SQS, and if a task is interrupted, the message becomes visible again after its visibility timeout and is retried, while a dead-letter queue (DLQ) catches jobs that keep failing. The service scales from 0 to N on queue depth, so nothing runs (or costs money) when the queue is empty. The trade-off is that scaling from zero takes a minute or two, which is acceptable for batch work but not for anything live.

However, in terms of managing cloud infrastructure, Lambda has the advantage in simplicity and in cost for low, irregular volumes, while Fargate Spot wins on runtime consistency and flexibility for longer or heavier jobs. Additionally, in the long run, the cost of Fargate Spot will be better than Lambda. So I chose Fargate Spot for the sim-worker, but if simulations are short and infrequent, the Redback team can keep the original Lambda. Fargate Spot should not be used for the telemetry-gateway, since it holds live WebSocket sessions that cannot tolerate interruption.